

서울과학기술대학교

AI COSS 하계 부트캠프

2주차 음성 인식 (ASR)

2026.07.14.

오울소프트 오유리
ceo@oul.kr

목차

- 실습환경: jupyter notebook
- ASR 실습

2026 하계 인공지능 SCI 부트캠프 일정					
	월	화	수	목	금
1주	6	7	8	9	10
			상상관 212호 10:30 기업전문가 특강		
2주	13	14	15	16	17
		상상관 212호 10:30 기업전문가 특강			
3주	20	21	22	23	24
		상상관 212호 10:30 기업전문가 특강		상상관 108호 10:30 중간평가(발표)	딥러닝실습 성적부여
4주	27	28	29	30	31
		상상관 212호 10:30 기업전문가 특강			상상관 108호 10:00 경진대회

1주차 과제 review

- AIHUB
 - 확인사항: 복지 데이터
- GITHUB
 - 계정id만 제공
 - 비밀번호: 제공한 경우 재설정 권장

실습환경: jupyter notebook

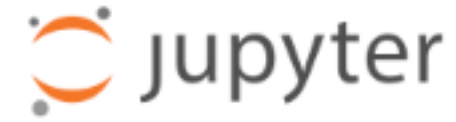
- Jupyter notebook?
 - 코드를 작성하고 실행 결과를 바로 가능한 학습 도구
 - AI, 데이터 분석, Python을 처음 실습할 때 적합

실습환경: jupyter notebook

- Jupyter notebook 장점
 - 코드를 실행하면 바로 결과가 보임
 - 한 줄씩, 단계별로 연습할 수 있음
 - 코드와 설명(note)을 같이 볼 수 있음
 - 실험하고 비교하기 좋음
 - 값을 조금 바꿔가며 여러 번 실행 가능
 - AI·데이터 학습에 잘 맞음
 - 데이터 확인, 그래프 출력, 모델 성능 비교 등을 한 화면에서 볼 수 있음
 - 과제 제출과 설명이 쉬움
 - 코드 + 결과 + 설명이 한 파일에 들어 있어 과제 제출, 발표, 복습에 모두 유용
- 주의사항
 - Jupyter Notebook은 공부와 실험을 위한 도구
 - 실제 서비스 개발이나 자동 실행(서버, 배치 작업)에서는 터미널(CLI) 방식이 더 적합

실습환경: jupyter notebook

- jupyter notebook 입장
 - 서버정보
 - 개별 제공
 - IP, Port, 비밀번호
 - 웹브라우저에서 아래 주소 입력
 - `http://<IP>:<Port>/login`
 - Password 칸에 비밀번호 입력
 - Log In 버튼 클릭



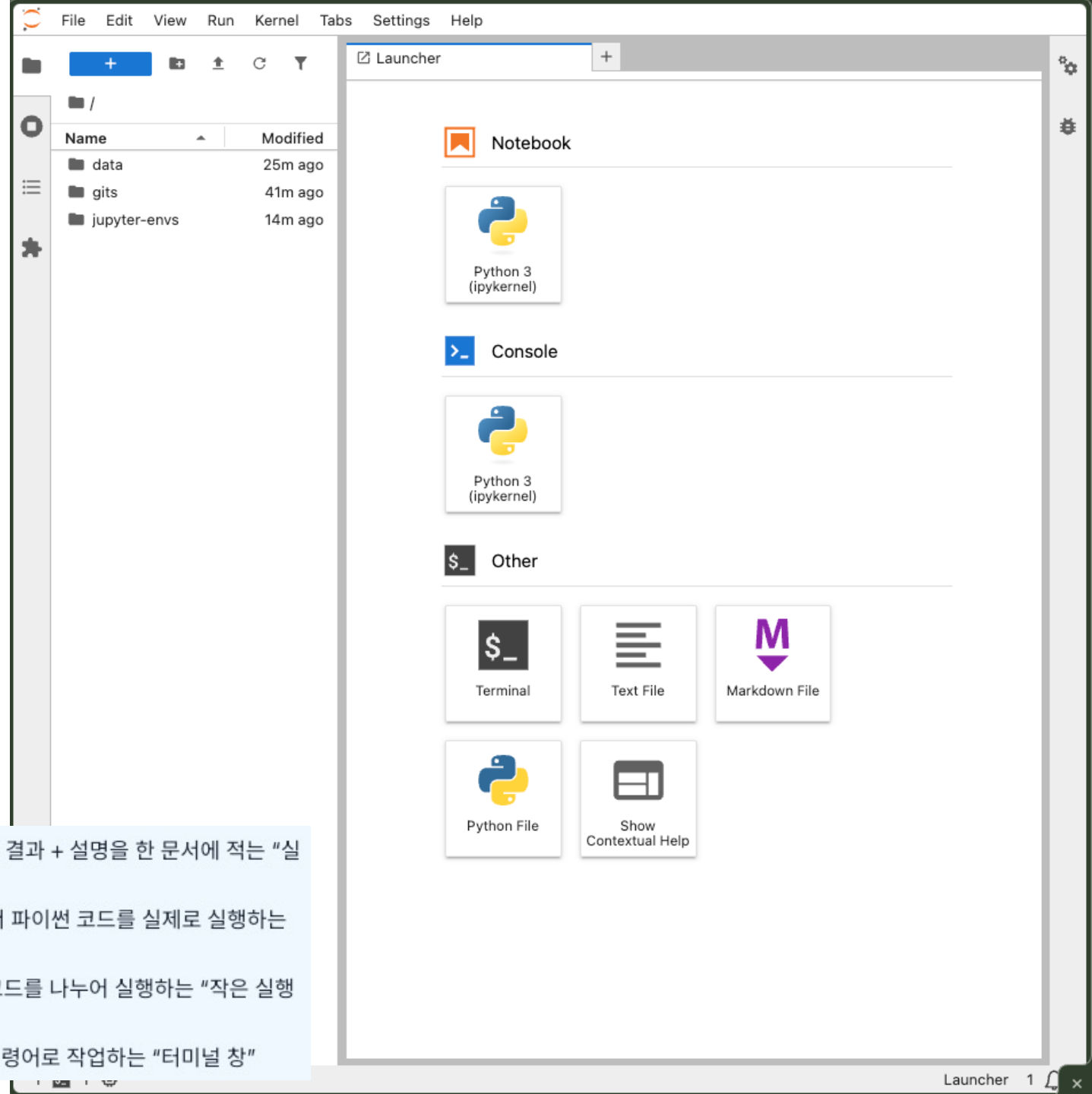
Password:

Log in

실습환경: jupyter notebook

- 왼쪽 파일 브라우저
 - 윈도우 탐색기 같은 역할
 - / : 최상위 폴더
- 가운데/오른쪽 Launcher
 - 새 작업을 여는 시작 메뉴
 - Notebook, Console, Terminal 등 선택
- 상단 메뉴(File / Edit / Run / Kernel ...)
 - 저장/실행/커널(파이썬 엔진) 관리를 하는 제어 메뉴
 - 주요 항목: Run(실행), Kernel(파이썬 엔진)

- **Notebook**: 코드 + 결과 + 설명을 한 문서에 적는 "실습 노트"
- **Kernel**: 노트북에서 파이썬 코드를 실제로 실행하는 "엔진"
- **Cell**: 노트북에서 코드를 나누어 실행하는 "작은 실행 단위"
- **Terminal(CLI)**: 명령어로 작업하는 "터미널 창"



실습환경: jupyter notebook

- 기본 폴더 구성
 - /data/aihub : 읽기 전용, aihub 데이터셋 공유
 - /data/zeroth : 읽기 전용, zeroth 데이터 공유
 - /gits : 읽기 전용, 수업에 활용할 자료
 - /jupyter-envs : jupyter notebook 커널

참고: 서버환경 확인

- 서버 환경 정보를 파악해 보는 명령어
- \$ nproc
- \$ nvidia-smi -L
- \$ free -h

```
user@1021794a749b:~$ nproc
17
user@1021794a749b:~$ nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-80GB (UUID: GPU-3669da9f-2d40-8319-bf4a-e0ca4a2bc8e8)
      MIG 3g.40gb      Device 0: (UUID: MIG-d46eec4e-7042-552d-8754-1adec59a2157)
user@1021794a749b:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	2.0Ti	11Gi	1.9Ti	107Mi	11Gi	1.9Ti
Swap:	0B	0B	0B			

```
user@1021794a749b:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
```

실습데이터: zeroth

- 공개 한국어 코퍼스
 - OpenSLR의 SLR40 (Zeroth-Korean)
 - <https://openslr.org/40/>
 - 라이선스: CC BY 4.0
 - 데이터셋을 공유/재사용/가공 가능
 - 결과물/문서/보고서에서 출처(저작자표시) 필요
- 용도
 - 한국어 자동음성인식(ASR) 학습/평가용 음성+전사 데이터
- 구성
 - 음성(약 10GB), 전사(transcript), lexicon, language model, 형태소 기반 분절기(morfessor)
- 규모
 - 학습데이터 51.6h(22,263발화, 105명)
 - 평가데이터 1.2h(457발화, 10명)

실습데이터: zeroth

- 서버 위치 /home/data/data/zeroth/
- Jupyter 위치 /data/zeroth/
 - train data: 학습용 데이터 (음성/전사파일)
 - **test data**: 평가용 데이터 (음성/전사파일)

OpenSLR

Open Speech and Language Resources

[Home](#) [Resources](#)

Zeroth-Korean

Identifier: SLR40

Summary: Korean Open-source Speech Corpus for Speech Recognition by Zeroth Project (<https://github.com/goodatlas/zeroth>)

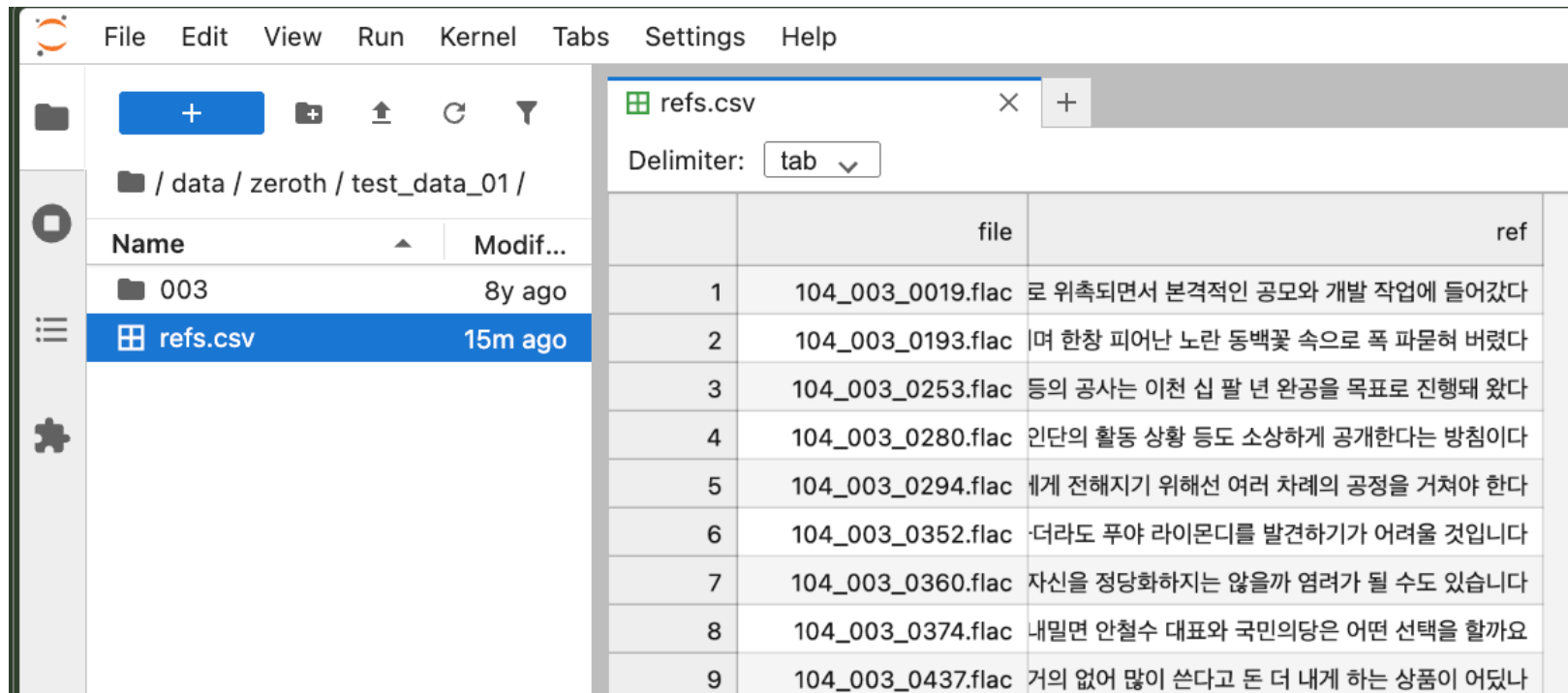
Category: Speech Corpus for Automatic Speech Recognition

License: Attribution 4.0 International (CC BY 4.0)

Downloads (use a mirror closer to you):
[zeroth_korean.tar.gz](#) [10G] (Korean Speech data, transcription, lexicon and language model) Mirrors: [\[EU\]](#) [\[EU\]](#) [\[CN\]](#)

실습데이터: zeroth

- 평가용 데이터 전사파일
 - csv 파일 (탭으로 구분)
 - 음성파일명 (file열), 전사텍스트(ref열)

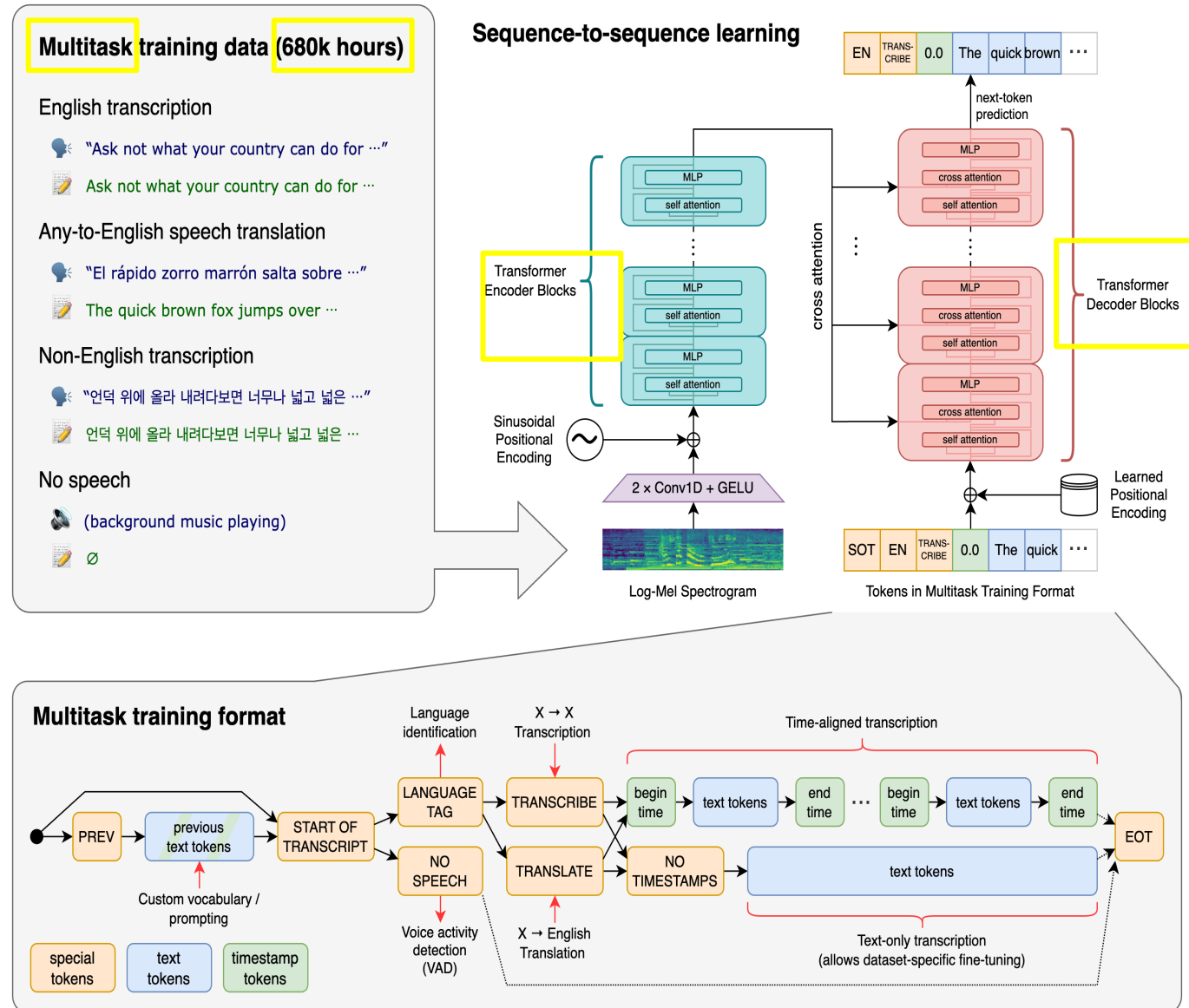


The screenshot shows a file editor interface with a menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help) and a toolbar. The left sidebar shows a file tree with the path `/ data / zeroth / test_data_01 /`. The main editor displays the file `refs.csv` with a tab delimiter. The CSV content is as follows:

	file	ref
1	104_003_0019.flac	로 위촉되면서 본격적인 공모와 개발 작업에 들어갔다
2	104_003_0193.flac	이며 한창 피어난 노란 동백꽃 속으로 폭 파묻혀 버렸다
3	104_003_0253.flac	등의 공사는 이천 십 팔 년 완공을 목표로 진행돼 왔다
4	104_003_0280.flac	인단의 활동 상황 등도 소상하게 공개한다는 방침이다
5	104_003_0294.flac	에게 전해지기 위해선 여러 차례의 공정을 거쳐야 한다
6	104_003_0352.flac	더라도 푸야 라이몬디를 발견하기가 어려울 것입니다
7	104_003_0360.flac	자신을 정당화하지는 않을까 염려가 될 수도 있습니다
8	104_003_0374.flac	내밀면 안철수 대표와 국민의당은 어떤 선택을 할까요
9	104_003_0437.flac	거의 없어 많이 쓴다고 돈 더 내게 하는 상품이 어딴나

실습ASR모델: whisper

- 2022년 9월 (오픈모델 공개)
 - <https://github.com/openai/whisper>
- 학습데이터
 - 약 68만 시간, 다양한 언어, 다양한 환경
- 지원언어
 - 약 100개 언어
- 기능
 - 음성 → 텍스트
 - 언어감지, 다국어 번역
 - 단어 시간 정보
 - 소음에 강인, 쉬운 파인튜닝
- 구조
 - Mel Spectrogram 음성 특징
 - Encoder-Decoder Transformer



실습ASR모델: whisper

- openAI Whisper 파인튜닝
 - 목적
 - 특정 도메인에 대한 성능 최적화
 - 지원 학습 방식
 - 전체 fine-tuning: 이번 주 진행할 실습
 - parameter-efficient tuning (LoRA, Prefix Tuning 등)

실습ASR모델: whisper

- Hugging Face 기반 음성인식 모델 예

	장점	단점	domain
Whisper (openai/whisper-*)	- 다국어 지원 (한국어 포함) - 소음에 강함 - 간단한 pipeline 사용 가능 - 자막 생성, 번역 기능 포함	- 비교적 무겁고 느림 (특히 large) - 실시간 처리엔 최적화 부족	- 일반 음성 인식 - 키오스크 - 자막 생성
Whisper Large-ko (kakao/whisper-large-ko)	- 한국어 특화 파인튜닝 - Whisper보다 한국어 정확도 높음	- Hugging Face에서만 사용 가능 - 학습 데이터 제한적	- 한국어 키오스크 - 한국어 대화 시스템
Wav2Vec2 (facebook/wav2vec2-*)	- 경량 모델 존재 - 빠른 처리 가능 - 영어 음성 정확도 우수	- 한국어 성능 미흡 - 소음에 약함 - 긴 문장 인식 한계	- 영어 기반 시스템 - 간단한 데모용
HuBERT (facebook/hubert-*)	- SSL 기반으로 다양한 언어 처리 가능성 - 연구 목적에 적합	- 사전학습된 모델 적음 - 실사용 예제 부족	- 연구용 / 실험적 학습
SeamlessM4T (meta)	- STT + 번역 통합 - Whisper 대체 가능성 - Hugging Face 일부 지원	- 모델 크고 느림 - 실시간 적합하지 않음	- 번역 키오스크 - 다국어 처리

ASR 실습 (Whisper, zeroth Korean)-1

- 위치: `gits/bootcamp-202607/exprs`
- 파일명: `01-asr_whisper_baseline_zeroth.invnh`
 - Zeroth Korean 데이터셋을 대상
 - Whisper ASR 베이스라인 성능 확인
- 커널: Python 3 (week-2 ASR)
- 입력 오디오
 - flac 형식의 음성데이터
- 정답 스크립트
 - tab 으로 구분된 csv 파일

Install packages

- > Import modules
- > Set paths and parameters
- > Select GPU/CPU and build ASR pipeline
- > Collect WAV files
- > Transcribe in batches
- > Load refs.csv and normalize keys
- > Merge ref and hyp by filename

Keyboard shortcut

- **Shift + Enter**: 현재 셀 실행 후 다음 셀로 이동
- **Ctrl + Enter**: 현재 셀만 실행(다음 셀로 이동하지 않음)

ASR 실습 (Whisper, zeroth Korean)-2

- 위치: `gits/bootcamp-202607/exprs`
- 파일명: `02-asr_whisper_baseline_zeroth-compare.ipynb`
 - Zeroth Korean 데이터셋을 대상
 - **Whisper 모델별 성능(WER/CER) 및 실행시간 저장**
- 커널: Python 3 (week-2 ASR)
- 입력 오디오
 - flac 형식의 음성데이터
- 정답 스크립트
 - tab 으로 구분된 csv 파일

Install packages

→ Import libraries

→ Set paths and experiment parameters (models, seed, output)

→ Load and normalize refs.csv (file, ref)

→ Collect wav files and sample deterministically (SEED, MAX_FILES)

→ For each model: build pipeline → transcribe → merge → compute WER/CER → save files

→ Combine model outputs into a comparison table (wide format)

실무 감각 포인트

- **device/dtype**를 함수 내부에서 확정하여 "실행 환경" 영향을 최소화
- **temperature=0.0**로 전사 결과의 일관성을 높임
- **elapsed**를 측정하여 "속도"도 같이 기록

ASR 실습 (Whisper, zeroth Korean)-2

- 결과 파일 예시

```
outputs_asr/  
  hyp_openai_whisper-tiny.csv  
  eval_openai_whisper-tiny.csv  
  hyp_openai_whisper-base.csv  
  eval_openai_whisper-base.csv  
  hyp_openai_whisper-small.csv  
  eval_openai_whisper-small.csv  
  benchmark_results.csv (선택적으로 저장/확장 가능)
```

5) 자주 겪는 문제(실습용 빠른 조치)

1. 커널이 멈춘 것 같아요 / 실행이 너무 오래 걸려요

- 상단 메뉴에서 **Kernel** → **Interrupt Kernel**로 중단
- 필요 시 **Kernel** → **Restart Kernel**로 재시작
- **MAX_FILES** 를 더 줄여(예: 10) 빠르게 검증 후 다시 늘리기

2. 데이터 경로가 없다고 나와요 (Missing audio dir / Missing refs.csv)

- **~/data_zeroth** 아래에 **test_data_01**, **refs.csv** 가 있는지 확인
- 심볼릭 링크가 제공되는 환경이면, 링크가 생성되었는지 확인

3. 모델 다운로드/로딩이 실패해요

- 네트워크 상태/캐시 문제일 수 있어, 잠시 후 다시 실행
- 환경에 따라 Hugging Face 캐시 디렉토리 정책이 다를 수 있음

ASR 실습 (Whisper, zeroth Korean)-3

- 03-prepare dataset for training.ipynb
 - ASR 모델 튜닝을 위한 데이터 준비용 노트북
- 05-record-audio.ipynb
 - 음성 녹음용 노트북
- 07-prepare recorded dataset for training.ipynb
 - 녹음한 데이터를 이용한 ASR 모델 튜닝을 위한 데이터 준비용 노트북
- 08-whisper_finetune_recorded.ipynb
 - 음성데이터를 이용한 ASR 모델용 노트북

이번주 목표: 팀원 음성을 위한 음성인식모델 튜닝



음성 인식 (ASR) – 실습

whisper 모델 파인튜닝

개요

- **목표**
 - 한국어 인식에 특화된 Whisper 모델 생성
- **세부 목표**
 - OpenAI의 Whisper 모델 사용
 - Mozilla Common Voice 데이터 활용
 - Hugging Face 기반 음성 모델 파이프라인 실습

개요: Whisper 소개

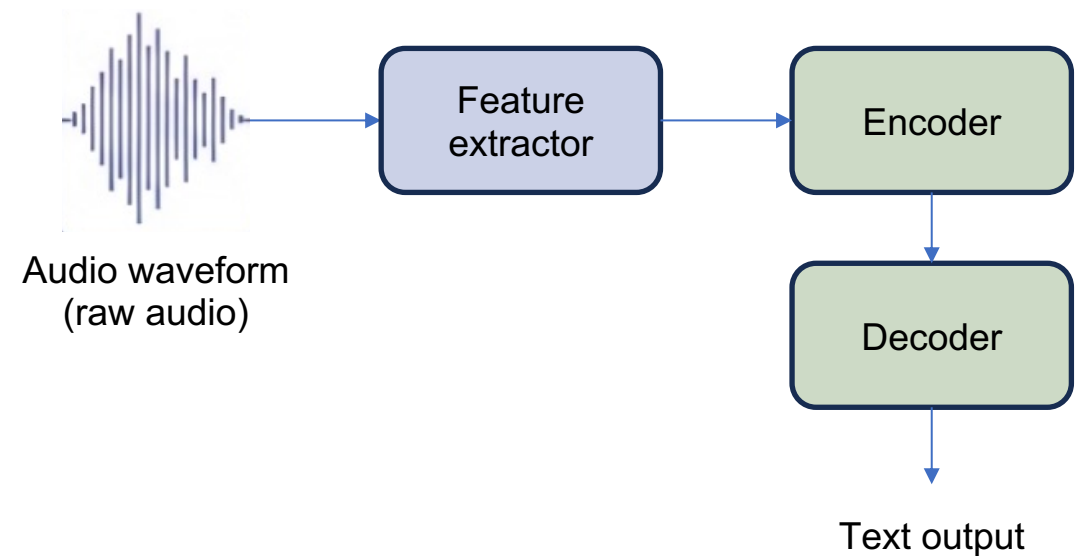
- **Whisper는?**

- OpenAI에서 공개한 다국어 음성 인식 모델
- 음성에서 텍스트로의 변환에 특화됨
- Sequence-to-Sequence (Seq2Seq) 구조

- **Whisper 특징**

- 다양한 언어 인식 가능 (약 100개 언어 지원)
- 노이즈에 강함
- 텍스트 변환, 언어번역 등 기능 제공
- 사전 학습된 모델 공개
 - tiny, base, small, medium, large

- **Whisper 모델 주요 구조**



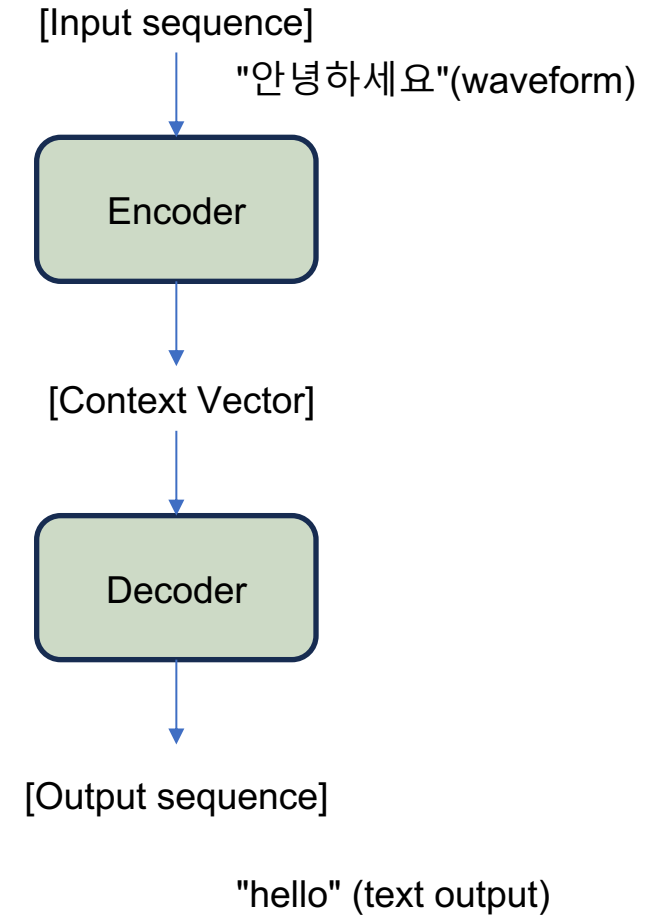
개념 설명: Sequence-to-Sequence (Seq2Seq)

- **Seq2Seq 정의**

- 하나의 시퀀스(입력)을 받아 다른 시퀀스(출력)로 바꾸는 모델 구조

- **Seq2Seq 특징**

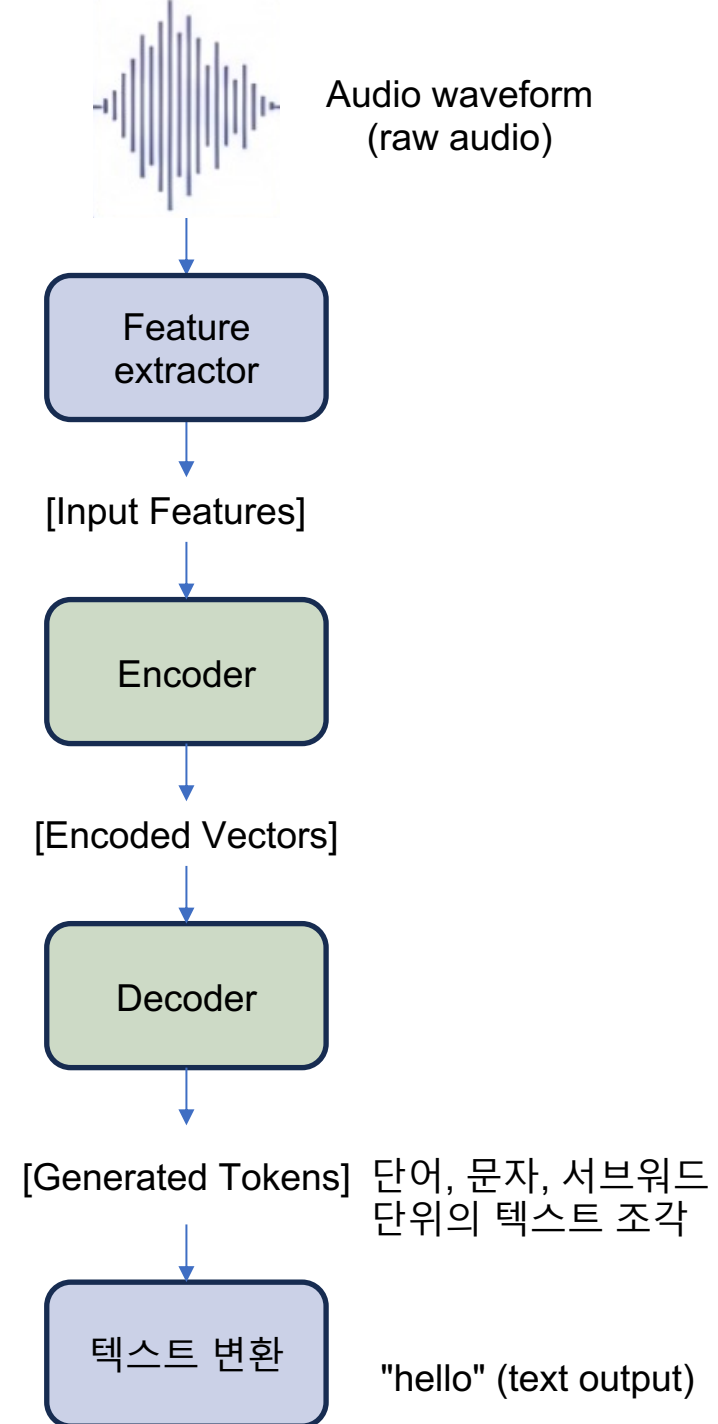
- 입력과 출력 길이가 다를 수 있음
 - 음성 → 텍스트
 - 번역
- 일반적으로 Encoder와 Decoder 로 구성



개념 설명: Whisper의 구조

- **Whisper의 구조**

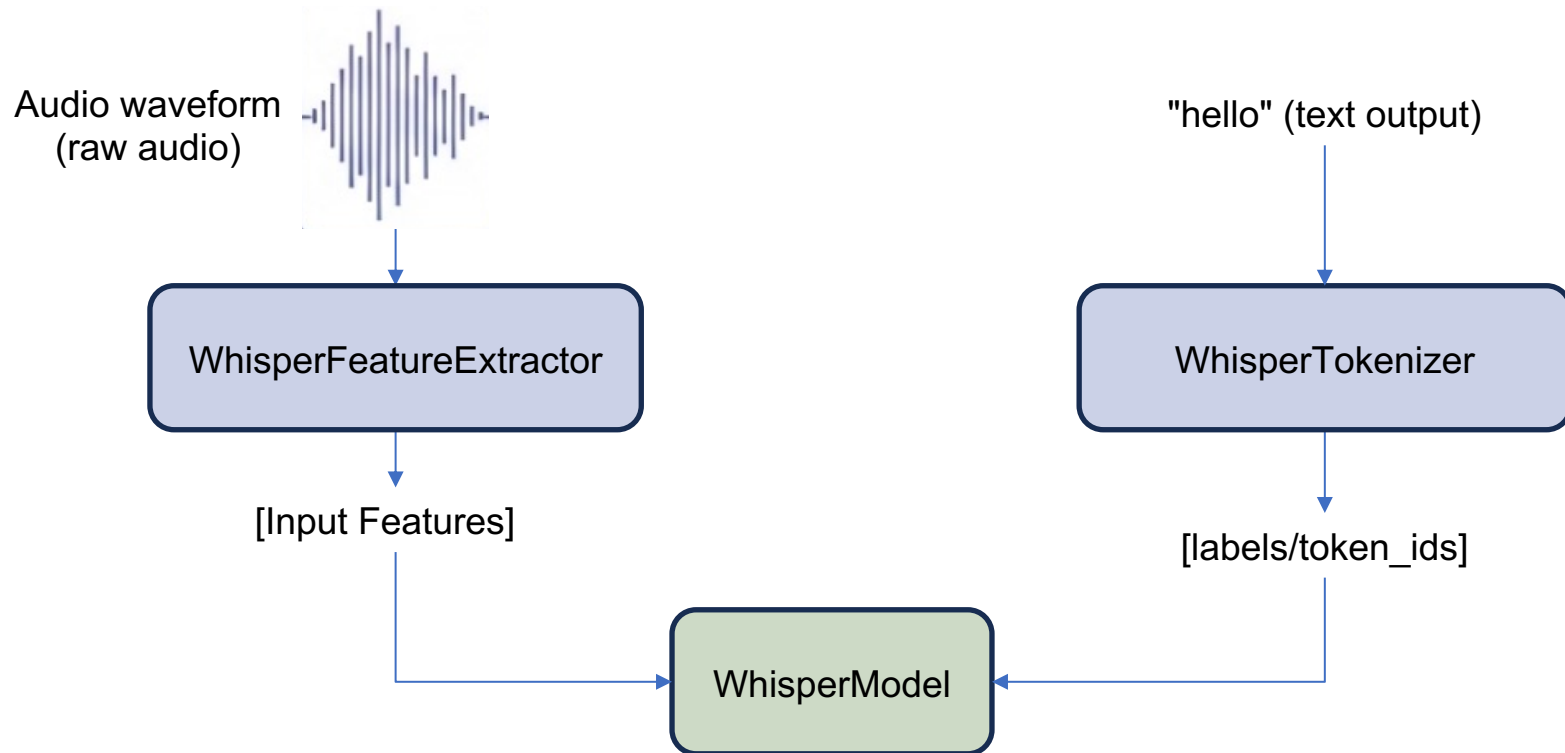
- Encoder-Decoder 기반
- Seq2Seq 방식과 잘 맞음



개념 설명: WhisperProcessor

- **WhisperProcessor?**

- Whisper 모델의 입출력 전처리를 담당
- 다음을 포함함
 - WhisperFeatureExtractor: waveform $\rightarrow$ input_features
 - WhisperTokenizer: 텍스트 $\leftrightarrow$ token id



개념 설명: WhisperForConditionalGeneration

- **WhisperForConditionalGeneration?**

- 실제 음성 인식(텍스트 생성)을 수행하는 Whisper 모델
- Encoder-Decoder 기반 모델
- Training Mode (학습 시)
 - input_features, labels를 입력 → loss 반환

항목	입출력	설명
input_features	입력	오디오 → Mel spectrogram (WhisperProcessor로 처리된 음성 입력)
labels	입력	정답 문장의 텍스트 (예: "안녕하세요")를 토큰화한 것
.loss	출력	모델이 예측한 값과 실제 정답(labels) 사이의 오차 값 (Loss)

- Inference / Generate Mode (추론 시)
 - generate() 사용 → 텍스트 토큰 시퀀스 반환

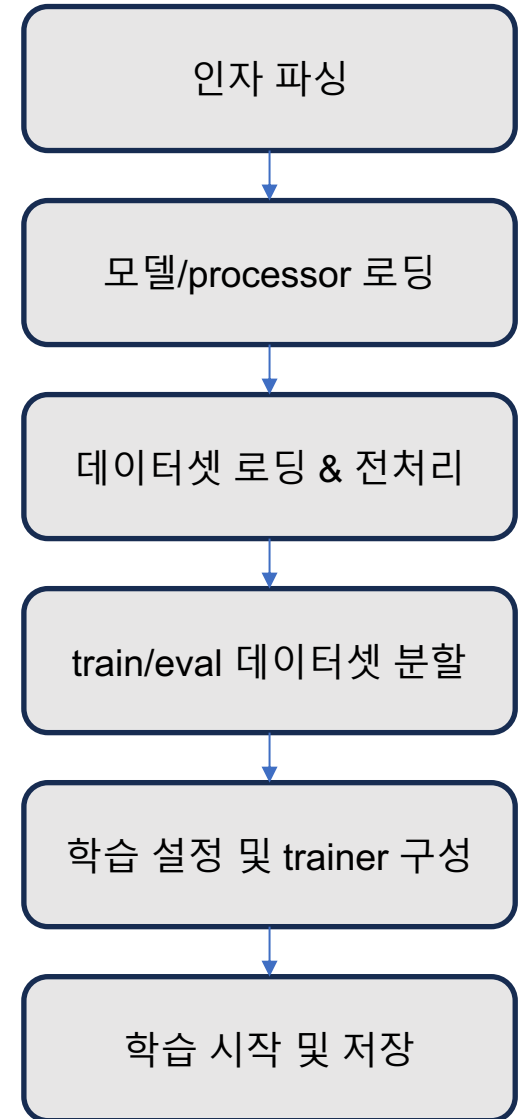
항목	입출력	설명
input_features	입력	오디오 → Mel spectrogram (WhisperProcessor로 처리된 음성 입력)
텍스트 토큰 시퀀스	출력	[50257, 416, 8573, ...] 형태의 토큰 ID 나열. 문장으로 복원 가능

코드 구조: 전체 학습 흐름 제어

- 역할

- 모델 로딩 및 데이터셋 전처리
- 학습/검증 데이터 분할
- Seq2SeqTrainer로 학습 수행

- 주요 흐름



Trainer 구성 및 학습 설정

- Hugging Face에서의 기본 학습 흐름



Trainer 구성 및 학습 설정: Seq2SeqTrainingArguments

- 주요 소스코드

```
from transformers import Seq2SeqTrainingArguments
```

```
training_args = Seq2SeqTrainingArguments(  
    output_dir="./outputs/whisper-ko",  
    per_device_train_batch_size=4,  
    learning_rate=1e-5,  
    max_steps=1000,  
    eval_steps=500,  
    save_steps=500,  
    logging_dir="./logs/whisper-ko",  
    report_to=["tensorboard"],  
    fp16=True  
)
```

인자	설명
output_dir	모델 저장 경로
per_device_train_batch_size	GPU/CPU당 배치 크기
learning_rate	학습률
max_steps	학습 반복 횟수
eval_steps	검증 주기
save_steps	체크포인트 저장 주기
fp16	mixed precision 학습 사용 여부 (GPU)

Trainer 구성 및 학습 설정: Seq2SeqTrainer

- 주요 소스코드

```
from transformers import Seq2SeqTrainer
```

```
trainer = Seq2SeqTrainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=eval_dataset,  
    tokenizer=processor.tokenizer,  
    data_collator=data_collator,  
)
```

인자	설명
model	Whisper 모델 (fine-tune 대상)
args	위에서 설정한 training_args
train_dataset / eval_dataset	학습 및 검증용 데이터셋
tokenizer	디코딩 시 사용
data_collator	배치 패딩 처리용 함수

Trainer 구성 및 학습 설정: 학습 시작 및 결과 저장

- 주요 소스코드

```
trainer.train()
```

```
model.save_pretrained(output_dir)
```

```
processor.save_pretrained(output_dir)
```

- 역할

- 모델과 processor가 output_dir 경로에 저장됨
- 추론 시 이 경로에서 불러와 바로 사용 가능

